Lab Handout 1-H
Field-Programmable Gate Array (FPGA) Universal Asynchronous Receiver/Transmitter (UART) Ingestion and End-to-End Data Flow

Capstone — Module 1: Radio Frequency (RF) Hardware Foundations

1 Objective

Wire the Nexys A7-100T to the STM32 interceptor (LH1-G), configure pin constraints, write the five VHSIC Hardware Description Language (VHDL) source files for a minimal end-to-end ingestion path, synthesize and program, and validate the complete Module 1 data flow: **cooperative target → RF link → interceptor (Threat) → FPGA**.

Scope of the VHDL you write here

The VHDL listings in this handout are deliberately minimal — just enough to light up the seven-segment display from a live frame stream. They are *bring-up* versions: no Cyclic Redundancy Check (CRC) checking, no reject path, no feature extraction. Module 2 (LH2-B through LH2-H) replaces every block in this handout with a fully specified production version. Most blocks (`uart_rx`, `byte_fifo`, `seg7_driver`) keep the same entity and are edited in place; `frame_detector` is genuinely deleted and replaced by a different entity (`pkt_parser`) in LH2-D. Type the listings in verbatim — do not skip steps trying to "get ahead" of the curriculum.

2 Prerequisites

- Lab Handouts 1-E (Vivado toolchain), 1-F (cooperative target firmware), and 1-G (interceptor / Threat firmware) complete.
- The interceptor node from LH1-G must be producing valid frames on `USART3 TX` (`PC4`), with LH1-G Step 7's behavioral validation complete (USART2 console shows `# rssi[ID=...]` and `# target:` snapshots, plus `PAUSE -> ID=...` bursts on a button hold).
- LH1-E loopback projects *do not* need to be open; this handout creates a separate Vivado project (`ingest_top`) that lives alongside them.

3 Architecture

Block	Function
`uart_rx`	115200 8N1 UART receiver (16× oversampled)
`byte_fifo`	16-deep × 8-bit synchro. First In, First Out (FIFO) between `uart_rx` and frame detector
`frame_detector`	Synchronize on `0x7E` start, count 43 bytes, expose Beacon ID
`seg7_driver`	Drive the four right-most digits with the Beacon ID (hex) and a 16-bit frame counter
`ingest_top`	Top-level wiring; instantiates all four blocks

What Module 2 replaces

`uart_rx` is edited in place in LH2-B (generics added, port list unchanged); `byte_fifo` is edited in place in LH2-C (depth widened from 16 to 64); `frame_detector` is *deleted* and replaced by a different entity, `pkt_parser`, in LH2-D; `seg7_driver` stays unchanged through LH2-D (only `ingest_top`'s wiring around it changes) and is finally retired in LH2-F when the seven-segment is replaced by feature-vector readout. `ingest_top` is edited in place in LH2-D, LH2-E, and LH2-F to track the changing block lineup. Knowing in advance which files are stable across modules and which are short-lived is part of why the listings below are kept minimal.

4 Wiring

Connect the Threat's `USART3 TX` pin to one of the Nexys A7 PMOD input pins. Common ground is essential.

From (Threat Nucleo)	Function	To (Nexys A7)	FPGA Pin
Morpho `PC4` (USART3_TX)	3.3 V serial out, 115200 8N1	PMOD JA pin 1 (`JA[0]`)	`C17`
Morpho `GND`	Common ground	PMOD JA pin 5 (or 11)	—

[Figure placeholder]

`LH1H_pmod_wiring.png`

STM32 `USART3 TX` (`PC4`) wired to Nexys A7 PMOD JA pin 1 (FPGA pin `C17`, called `JA[0]` in the master XDC) with common ground on PMOD JA pin 5. Annotated photo clarifies which PMOD connector is in use.

Figure 1: STM32 `USART3 TX` (`PC4`) wired to Nexys A7 PMOD JA pin 1 (FPGA pin `C17`, called `JA[0]` in the master XDC) with common ground on PMOD JA pin 5. Annotated photo clarifies which PMOD connector is in use.

Voltage Levels and Connector Choice

The STM32 drives 3.3 V logic and the Nexys A7 PMOD pins are 3.3 V native. No level shifter is required. *Use PMOD JA, not JC* — some Nexys A7 board revisions wire JC (the high-speed PMOD next to the FMC connector) to a different I/O bank, and the LVDS-style pin pairs there have additional series resistors that can corrupt single-ended UART. JA, JB, and JD are all safe.

5 Procedure

5.1 Step 1 — Create the Vivado project

Step 1.1 — Launch Vivado and start a new project

1. Launch Vivado from the Start Menu (Windows) or the desktop launcher (Linux). The default Vivado start page opens.
2. Click **Quick Start** → **Create Project**. The New Project wizard opens.
3. Click **Next** on the wizard's first page.
4. On the **Project Name** page:
   - Project name: `ingest_top`
   - Project location: a local-only folder *not* inside OneDrive, iCloud Drive, Dropbox, or Google Drive (Vivado writes hundreds of intermediate files; cloud sync corrupts them mid-write).
   - Tick **Create project subdirectory**.

   Click **Next**.
5. On the **Project Type** page, select **RTL Project**. Tick **Do not specify sources at this time** (you will add them in Step 2). Click **Next**.
6. On the **Default Part** page, click the **Boards** tab (not Parts), type `Nexys A7-100T` in the search box, select the Digilent Nexys A7-100T row, and click **Next**.
   - If the board is not listed, your Vivado install is missing the Digilent board files. Stop, install them per the LH1-E Step 1 *Common Pitfalls* entry, and restart this handout.
   - Confirm the right-hand **Selected Part** field shows `xc7a100tcsg324-1`.
7. On the **New Project Summary** page, click **Finish**.

Step 1.2 — Verify the project opened correctly

1. The Vivado IDE should open with the Project Manager view active. The **Sources** panel (left side) is empty under *Design Sources*, *Constraints*, and *Simulation Sources*.
2. Confirm the title bar reads `ingest_top - [<your-path>/ingest_top.xpr] - Vivado <version>`.
3. Confirm the bottom-right of the IDE shows part `xc7a100tcsg324-1`.

5.2 Step 2 — Create the five VHDL source files

You will create five empty VHDL files, then fill each one with the listing below. Do them in order; each later file depends on the entity declarations of the earlier ones.

Step 2.1 — Create empty VHDL files in Vivado

1. In the Sources panel, click + → **Add or create design sources** → **Create File**.
2. File type: **VHDL**. File name: `uart_rx`. Click **OK**, then **Finish**, then **OK** on the "Define Module" dialog without adding any ports (you will overwrite the entire file in Step 2.2). If asked "Yes, continue" to a warning about no ports, click **Yes**.
3. Repeat for the four remaining files in this exact order: `byte_fifo`, `frame_detector`, `seg7_driver`, `ingest_top`.
4. In the Sources panel under *Design Sources* you should now see all five files.

Step 2.2 — Fill in `uart_rx.vhd`

Double-click `uart_rx` in the Sources panel. Vivado opens an auto-generated stub. Select all (`Ctrl+A`), delete, and paste the listing below verbatim. Save (`Ctrl+S`).

```vhdl
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity uart_rx is
    port (
        clk        : in  std_logic;                        -- 100 MHz sysclk
        rst_n      : in  std_logic;                        -- active-low reset
        rx         : in  std_logic;                        -- async serial input
        rx_data    : out std_logic_vector(7 downto 0);     -- received byte
        rx_strobe  : out std_logic                         -- 1-cycle pulse on rx_data valid
    );
end uart_rx;

architecture rtl of uart_rx is
    constant DIV16 : integer := 100_000_000 / (115_200 * 16);  -- = 54

    signal os_cnt  : unsigned(5 downto 0) := (others => '0');
    signal os_tick : std_logic := '0';

    type state_t is (S_IDLE, S_START, S_DATA, S_STOP);
    signal state    : state_t := S_IDLE;
    signal bit_idx  : unsigned(2 downto 0) := (others => '0');
    signal sub_cnt  : unsigned(3 downto 0) := (others => '0');
    signal shifter  : std_logic_vector(7 downto 0) := (others => '0');
    signal rx_sync1, rx_sync2 : std_logic := '1';
begin
    -- Two-flop synchronizer for the async rx pin (idle-high reset value
    -- prevents a false start bit during reset release).
    process (clk) begin
        if rising_edge(clk) then
            rx_sync1 <= rx;
            rx_sync2 <= rx_sync1;
        end if;
    end process;

    -- 16x oversample tick generator (one-cycle pulse every DIV16 sysclks).
    process (clk) begin
        if rising_edge(clk) then
            if rst_n = '0' then
                os_cnt  <= (others => '0');
                os_tick <= '0';
            elsif os_cnt = to_unsigned(DIV16 - 1, os_cnt'length) then
                os_cnt  <= (others => '0');
                os_tick <= '1';
            else
                os_cnt  <= os_cnt + 1;
                os_tick <= '0';
            end if;
        end if;
    end process;

    -- Receive FSM, advanced on every os_tick.
    process (clk) begin
        if rising_edge(clk) then
            rx_strobe <= '0';
            if rst_n = '0' then
                state   <= S_IDLE;
                sub_cnt <= (others => '0');
                bit_idx <= (others => '0');
            elsif os_tick = '1' then
                case state is
                    when S_IDLE =>
```

```
                        if rx_sync2 = '0' then            -- start bit edge
                            state   <= S_START;
                            sub_cnt <= (others => '0');
                        end if;
                    when S_START =>
                        if sub_cnt = to_unsigned(7, 4) then   -- mid-bit sample
                            if rx_sync2 = '0' then
                                state   <= S_DATA;
                                sub_cnt <= (others => '0');
                                bit_idx <= (others => '0');
                            else
                                state <= S_IDLE;            -- glitch, abort
                            end if;
                        else
                            sub_cnt <= sub_cnt + 1;
                        end if;
                    when S_DATA =>
                        if sub_cnt = to_unsigned(15, 4) then
                            shifter <= rx_sync2 & shifter(7 downto 1);  -- LSB first
                            sub_cnt <= (others => '0');
                            if bit_idx = to_unsigned(7, 3) then
                                state <= S_STOP;
                            else
                                bit_idx <= bit_idx + 1;
                            end if;
                        else
                            sub_cnt <= sub_cnt + 1;
                        end if;
                    when S_STOP =>
                        if sub_cnt = to_unsigned(15, 4) then
                            rx_data   <= shifter;
                            rx_strobe <= '1';
                            state     <= S_IDLE;
                        else
                            sub_cnt <= sub_cnt + 1;
                        end if;
                end case;
            end if;
        end if;
    end process;
end rtl;
```

Listing 1: `uart_rx.vhd` — 115200 8N1 receiver, 16× oversampled. Bring-up version. Module 2 LH2-B replaces this with a generic-parameterized version and adds a unit testbench.

Step 2.3 — Fill in `byte_fifo.vhd`

Open `byte_fifo`, replace its entire contents with the listing below, and save.

```vhdl
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity byte_fifo is
    port (
        clk     : in  std_logic;
        rst_n   : in  std_logic;
        wr_en   : in  std_logic;
        wr_data : in  std_logic_vector(7 downto 0);
        rd_en   : in  std_logic;
        rd_data : out std_logic_vector(7 downto 0);
        empty   : out std_logic;
        full    : out std_logic
    );
end byte_fifo;

architecture rtl of byte_fifo is
    constant DEPTH : integer := 16;
    type mem_t is array (0 to DEPTH - 1) of std_logic_vector(7 downto 0);
    signal mem   : mem_t := (others => (others => '0'));
    signal w_ptr : integer range 0 to DEPTH - 1 := 0;
    signal r_ptr : integer range 0 to DEPTH - 1 := 0;
    signal count : integer range 0 to DEPTH     := 0;
begin
    full  <= '1' when count = DEPTH else '0';
    empty <= '1' when count = 0     else '0';

    process (clk)
        variable do_wr, do_rd : boolean;
    begin
        if rising_edge(clk) then
            if rst_n = '0' then
                w_ptr <= 0; r_ptr <= 0; count <= 0;
            else
                do_wr := (wr_en = '1') and (count < DEPTH);
                do_rd := (rd_en = '1') and (count > 0);
                if do_wr then
                    mem(w_ptr) <= wr_data;
                    w_ptr <= (w_ptr + 1) mod DEPTH;
                end if;
                if do_rd then
                    r_ptr <= (r_ptr + 1) mod DEPTH;
                end if;
                if    do_wr and not do_rd then count <= count + 1;
                elsif do_rd and not do_wr then count <= count - 1;
                end if;
            end if;
            rd_data <= mem(r_ptr);    -- registered read; valid 1 cycle after rd_en
        end if;
    end process;
end rtl;
```

Listing 2: `byte_fifo.vhd` — 16-entry × 8-bit synchronous FIFO. Bring-up depth is sufficient because the frame detector drains continuously. LH2-C widens to 64.

Step 2.4 — Fill in `frame_detector.vhd`

Open `frame_detector`, replace its entire contents with the listing below, and save.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity frame_detector is
    port (
        clk         : in  std_logic;
        rst_n       : in  std_logic;
        -- byte_fifo read interface
        fifo_empty  : in  std_logic;
        fifo_data   : in  std_logic_vector(7 downto 0);
        fifo_rd_en  : out std_logic;
        -- Outputs
        beacon_id   : out std_logic_vector(7 downto 0);  -- frame byte 12
        new_frame   : out std_logic                       -- 1-cycle pulse on full frame
    );
end frame_detector;

architecture rtl of frame_detector is
    type state_t is (S_HUNT, S_REQ, S_WAIT, S_CAPTURE);
    signal state     : state_t := S_HUNT;
    signal byte_idx  : integer range 0 to 63 := 0;
    signal beacon_r  : std_logic_vector(7 downto 0) := (others => '0');
begin
    beacon_id <= beacon_r;

    process (clk) begin
        if rising_edge(clk) then
            new_frame  <= '0';
            fifo_rd_en <= '0';

            if rst_n = '0' then
                state    <= S_HUNT;
                byte_idx <= 0;
                beacon_r <= (others => '0');
            else
                case state is
                    when S_HUNT =>
                        -- Pop bytes one at a time until we see 0x7E.
                        if fifo_empty = '0' then
                            fifo_rd_en <= '1';
                            state      <= S_WAIT;
                            byte_idx   <= 0;
                        end if;
                    when S_REQ =>
                        -- Issue a read for the next byte of the frame.
                        if fifo_empty = '0' then
                            fifo_rd_en <= '1';
                            state      <= S_WAIT;
                        end if;
                    when S_WAIT =>
                        -- One-cycle latency: rd_data is valid this cycle.
                        state <= S_CAPTURE;
                    when S_CAPTURE =>
                        if byte_idx = 0 then
                            -- Hunting for the start delimiter.
                            if fifo_data = x"7E" then
                                byte_idx <= 1;
                                state    <= S_REQ;
                            else
                                state <= S_HUNT;          -- still hunting
                            end if;
                        else
                            -- Frame in progress.
```

```
                            if byte_idx = 12 then
                                beacon_r <= fifo_data;     -- radio packet byte 2
                            end if;
                            if byte_idx = 42 then
                                new_frame <= '1';
                                state     <= S_HUNT;
                                byte_idx  <= 0;
                            else
                                byte_idx <= byte_idx + 1;
                                state    <= S_REQ;
                            end if;
                        end if;
                end case;
            end if;
        end if;
    end process;
end rtl;
```

Listing 3: `frame_detector.vhd` — bring-up frame detector. Locks on `0x7E`, captures Beacon ID at frame byte 12 (radio packet byte 2), counts to 43, and pulses `new_frame`. No CRC check, no reject path. **Deleted in LH2-D and replaced by `pkt_parser`.**

Step 2.5 — Fill in `seg7_driver.vhd`

Open `seg7_driver`, replace its entire contents with the listing below, and save.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity seg7_driver is
    port (
        clk        : in  std_logic;                       -- 100 MHz
        rst_n      : in  std_logic;
        beacon_id  : in  std_logic_vector(7 downto 0);
        new_frame  : in  std_logic;
        seg        : out std_logic_vector(6 downto 0);    -- CA..CG, active low
        dp         : out std_logic;
        an         : out std_logic_vector(7 downto 0)     -- 8 anodes, active low
    );
end seg7_driver;

architecture rtl of seg7_driver is
    -- 17-bit refresh counter: top 2 bits select the digit (~750 Hz/digit at 100 MHz).
    signal refresh_cnt : unsigned(16 downto 0) := (others => '0');
    signal digit_sel   : unsigned(1 downto 0);

    signal frame_cnt   : unsigned(15 downto 0) := (others => '0');
    signal nibble      : std_logic_vector(3 downto 0);
    signal seg_n       : std_logic_vector(6 downto 0);
begin
    dp <= '1';                                      -- decimal point off

    -- Refresh counter and frame counter.
    process (clk) begin
        if rising_edge(clk) then
            if rst_n = '0' then
                refresh_cnt <= (others => '0');
                frame_cnt   <= (others => '0');
            else
                refresh_cnt <= refresh_cnt + 1;
                if new_frame = '1' then
                    frame_cnt <= frame_cnt + 1;
                end if;
            end if;
```

```vhdl
        end if;
    end process;

    digit_sel <= refresh_cnt(refresh_cnt'high downto refresh_cnt'high - 1);

    -- Select which 4-bit nibble drives the active digit.
    -- AN[0] = right-most digit. Mapping:
    --   digit_sel=0 -> AN[0] = beacon_id(3 downto 0)        (low nibble of ID)
    --   digit_sel=1 -> AN[1] = beacon_id(7 downto 4)        (high nibble of ID)
    --   digit_sel=2 -> AN[2] = frame_cnt(3 downto 0)        (low nibble of counter low byte)
    --   digit_sel=3 -> AN[3] = frame_cnt(7 downto 4)        (high nibble of counter low byte)
    process (digit_sel, beacon_id, frame_cnt) begin
        case digit_sel is
            when "00"   => nibble <= beacon_id(3 downto 0);
            when "01"   => nibble <= beacon_id(7 downto 4);
            when "10"   => nibble <= std_logic_vector(frame_cnt(3 downto 0));
            when others => nibble <= std_logic_vector(frame_cnt(7 downto 4));
        end case;
    end process;

    -- Hex-to-7-segment lookup. seg_n is the active-LOW pattern (CA..CG).
    -- Segment bit order: seg_n(6 downto 0) = G F E D C B A.
    process (nibble) begin
        case nibble is
            when x"0" => seg_n <= "1000000";
            when x"1" => seg_n <= "1111001";
            when x"2" => seg_n <= "0100100";
            when x"3" => seg_n <= "0110000";
            when x"4" => seg_n <= "0011001";
            when x"5" => seg_n <= "0010010";
            when x"6" => seg_n <= "0000010";
            when x"7" => seg_n <= "1111000";
            when x"8" => seg_n <= "0000000";
            when x"9" => seg_n <= "0010000";
            when x"A" => seg_n <= "0001000";
            when x"B" => seg_n <= "0000011";
            when x"C" => seg_n <= "1000110";
            when x"D" => seg_n <= "0100001";
            when x"E" => seg_n <= "0000110";
            when others => seg_n <= "0001110";  -- F
        end case;
    end process;
    seg <= seg_n;

    -- Anode mux: active digit pulled low; all others high.
    process (digit_sel) begin
        an <= (others => '1');                  -- default all off
        case digit_sel is
            when "00"   => an(0) <= '0';
            when "01"   => an(1) <= '0';
            when "10"   => an(2) <= '0';
            when others => an(3) <= '0';
        end case;
    end process;
end rtl;
```

Listing 4: `seg7_driver.vhd` — four-digit time-multiplexed display. Right two digits show the Beacon ID (hex); left two digits show the low byte of a 16-bit `new_frame` counter. Active-low segments and active-low anodes (Nexys A7 convention).

Step 2.6 — Fill in `ingest_top.vhd`

Open `ingest_top`, replace its entire contents with the listing below, and save.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity ingest_top is
    port (
        CLK100MHZ    : in  std_logic;
        CPU_RESETN   : in  std_logic;                        -- active-low pushbutton
        uart_rx_pin  : in  std_logic;                        -- PMOD JA pin 1 (C17)
        CA, CB, CC, CD, CE, CF, CG : out std_logic;          -- 7-seg segments (active low)
        DP           : out std_logic;
        AN           : out std_logic_vector(7 downto 0);
        LED          : out std_logic_vector(15 downto 0)
    );
end ingest_top;

architecture rtl of ingest_top is
    signal rst_n      : std_logic;

    signal rx_data    : std_logic_vector(7 downto 0);
    signal rx_strobe  : std_logic;

    signal fifo_data  : std_logic_vector(7 downto 0);
    signal fifo_rd_en : std_logic;
    signal fifo_empty : std_logic;
    signal fifo_full  : std_logic;

    signal beacon_id  : std_logic_vector(7 downto 0);
    signal new_frame  : std_logic;

    signal seg_bus    : std_logic_vector(6 downto 0);
begin
    rst_n <= CPU_RESETN;                                     -- already active-low

    u_uart : entity work.uart_rx
        port map (clk => CLK100MHZ, rst_n => rst_n,
                  rx => uart_rx_pin,
                  rx_data => rx_data, rx_strobe => rx_strobe);

    u_fifo : entity work.byte_fifo
        port map (clk => CLK100MHZ, rst_n => rst_n,
                  wr_en => rx_strobe, wr_data => rx_data,
                  rd_en => fifo_rd_en, rd_data => fifo_data,
                  empty => fifo_empty, full => fifo_full);

    u_det : entity work.frame_detector
        port map (clk => CLK100MHZ, rst_n => rst_n,
                  fifo_empty => fifo_empty, fifo_data => fifo_data,
                  fifo_rd_en => fifo_rd_en,
                  beacon_id => beacon_id, new_frame => new_frame);

    u_seg : entity work.seg7_driver
        port map (clk => CLK100MHZ, rst_n => rst_n,
                  beacon_id => beacon_id, new_frame => new_frame,
                  seg => seg_bus, dp => DP, an => AN);

    -- Break out the segment bus to the named XDC ports (CA..CG).
    CA <= seg_bus(0);
    CB <= seg_bus(1);
    CC <= seg_bus(2);
    CD <= seg_bus(3);
    CE <= seg_bus(4);
    CF <= seg_bus(5);
    CG <= seg_bus(6);

```

```
    -- Status LEDs (the rest are forced low).
    LED(0)              <= rx_strobe;
    LED(1)              <= new_frame;
    LED(2)              <= fifo_empty;
    LED(3)              <= fifo_full;
    LED(15 downto 4)    <= (others => '0');
end rtl;
```

Listing 5: `ingest_top.vhd` — top-level wiring. Port names match the constraint file in Step 3 exactly. `LED[0]` blinks per byte; `LED[1]` blinks per frame; `LED[2]` = FIFO empty; `LED[3]` = FIFO full.

Step 2.7 — Set top and check the hierarchy

1. In the Sources panel, right-click `ingest_top` and select **Set as Top**. The entity name should now appear in bold.

2. Confirm the Sources hierarchy shows `ingest_top` at the root with `u_uart`, `u_fifo`, `u_det`, and `u_seg` as children. Any yellow "?" or red "!" marker indicates a syntax error in that file — fix before continuing.

5.3 Step 3 — Add and configure the constraint file

1. In the Sources panel, click + → **Add or create constraints** → **Add Files**. Navigate to your `Nexys-A7-100T-Master.xdc` from LH1-E Step 2, ensure **Copy sources into project** is ticked, click **Finish**.

2. Double-click the copied `Nexys-A7-100T-Master.xdc` under *Constraints* to open it.

3. Uncomment *exactly* the following groups of lines by removing the leading `#` on each. All other lines in the master XDC must remain commented out:

   - The **100 MHz system clock** entry for port `CLK100MHZ` (pin `E3`) and its associated `create_clock` line.
   - The **CPU reset** pushbutton entry for port `CPU_RESETN` (pin `C12`).
   - The **PMOD JA pin 1** entry — search for pin `C17`. The master XDC labels this port `JA[1]` (the file uses 1-based indexing). Uncomment the line, then change `JA[1]` to `uart_rx_pin` so the final uncommented line reads:
     `set_property -dict { PACKAGE_PIN C17  IOSTANDARD LVCMOS33 } [get_ports { uart_rx_pin }];`
   - All **seven-segment segment** entries: `CA` through `CG` (pins `T10`, `R10`, `K16`, `K13`, `P15`, `T11`, `L18`) and the `DP` line (pin `H15`).
   - All eight **anode** entries: `AN[0]` through `AN[7]`.
   - **LED[0] through LED[15]** — all sixteen LED entries.

   Save the XDC (`Ctrl+S`).

Pin numbers are board-revision specific

The port names listed above are correct for the Nexys A7-100T (Rev. B and later, the production silkscreen). Older A7-100T prototypes and the A7-50T variant differ on a handful of pins (notably the seg7 anodes). If any of the ports listed above do not appear as commented-out lines in your master XDC, your master XDC is for a different board revision — re-download from Digilent's reference page using the exact SKU printed on your board.

5.4 Step 4 — Synthesize, implement, and program

1. Flow Navigator → **Run Synthesis**. Wait for completion. Expect zero errors.
2. Flow Navigator → **Run Implementation**. Wait for completion. After implementation, click **Open Implemented Design** → **Report Timing Summary** and confirm WNS ≥ 0 ns and WHS ≥ 0 ns.
3. Flow Navigator → **Generate Bitstream**. Output: `ingest_top.bit`.
4. Connect the Nexys A7 via USB and power it on.
5. Open Hardware Manager → **Open Target** → **Auto Connect**.
6. Right-click the `xc7a100t` device → **Program Device**, point at `ingest_top.bit`, click **Program**. The DONE LED should illuminate.

5.5 Step 5 — End-to-end run

1. Confirm the wiring per the table on page 2: Nucleo Morpho `PC4` → PMOD JA pin 1, GND → PMOD JA pin 5.
2. Power on Target A and Target B (LH1-F). Confirm their USART2 logs show the ping-pong CSV lines.
3. Power on the Threat (LH1-G). Confirm its USART2 log shows `RX sender=01` and `RX sender=02` lines alternating once per second.
4. Observe the Nexys A7 seven-segment display. Right-most two digits should alternate `01` (Target A's DATA) and `02` (Target B's ACK) in step with each frame; left-most two digits should show a slowly-incrementing 16-bit frame counter (low byte). `LED[0]` (UART byte strobe) should flicker rapidly; `LED[1]` (new_frame pulse) should blink at the per-frame cadence; `LED[2]` (FIFO empty) should be on most of the time; `LED[3]` (FIFO full) should never light.
5. **Perturbation 1:** power off Target A. The display should lock to `02` for one frame and then go silent (Target B has nothing to ACK). Power Target A back on; alternating display should resume within 2 s.
6. **Perturbation 2:** hold the Threat user button (B1) for 3 s. The display should briefly fall silent (the targeted Target stops transmitting DATA, so the other has nothing to ACK), then resume alternating within ∼500 ms of release.

[Figure placeholder]

`LH1H_seg7_display.jpg`

Module 1 capstone shot: Nexys A7 seven-segment display showing a live Beacon ID captured from the full ***cooperative target*** → ***RF link*** → ***interceptor (Threat)*** → ***FPGA*** *chain.*

Figure 2: Module 1 capstone shot: Nexys A7 seven-segment display showing a live Beacon ID captured from the full **cooperative target** → **RF link** → **interceptor (Threat)** → **FPGA** chain.

6 Verification Checklist

- ☐ All five VHDL files created and filled with the listings in Step 2; `ingest_top` marked as Top.
- ☐ Synthesis, Implementation, and Bitstream generation complete with zero errors.
- ☐ WNS ≥ 0 ns and WHS ≥ 0 ns in the implementation timing report.
- ☐ DONE LED illuminates after programming.
- ☐ `LED[0]` flickers (UART bytes arriving), `LED[1]` pulses at ∼1 Hz (new_frame), `LED[3]` (FIFO full) never lights.
- ☐ Seven-segment display alternates `01` and `02` matching the live Beacon IDs.
- ☐ Powering Target A off then on causes the display to silence then resume within 2 s.
- ☐ Holding the Threat's user button suspends the alternating display; releasing it restores normal operation within ∼500 ms.

Troubleshooting

- **No display change, `LED[0]` dark:** no UART bytes are reaching the FPGA. Re-check the PMOD wire (JA pin 1, not pin 2) and the GND wire. Verify with a scope on the FPGA side of the jumper that you see 3.3 V LVCMOS-level transitions when the Threat is transmitting.
- **`LED[0]` flickers but `LED[1]` stays dark:** bytes are arriving but no frame ever assembles. Likely the FIFO write or read enable is wrong, or the FSM never sees `0x7E`. Add an ILA probe on `rx_data` and look for `0x7E` bytes; if absent, your Threat is not actually emitting frames (re-flash the Threat firmware and re-confirm LH1-G's USART2-console validation).
- **Display garbled / wrong digits:** segment-anode mapping inverted. The Nexys A7 segments and anodes are *both active-low*; double-check your XDC and the `seg_n` lookup table.
- **Display flickers unreadably:** multiplex rate too slow or too fast. The default 17-bit refresh counter gives ∼750 Hz/digit; if you reduced the counter width below 14 bits you may see flicker or ghosting.
- **Frame detector stuck (`LED[1]` never pulses) but `0x7E` bytes are visible on ILA:** length mismatch in the frame body — if the Threat is emitting frames longer or shorter than 43 bytes, the byte counter never reaches 42. Cross-check the Threat's `build_forwarding_frame` from LH1-G Step 6 against the Module 1 frame format (43 bytes, length byte = `0x29`).

Lab Handout 1-H Deliverable — and Module 1 Final Deliverable

1. All five VHDL source files (`uart_rx.vhd`, `byte_fifo.vhd`, `frame_detector.vhd`, `seg7_driver.vhd`, `ingest_top.vhd`).
2. Pruned `Nexys-A7-100T-Master.xdc` with only the active lines shown in Step 3 uncommented.
3. Vivado utilization and timing reports (raw `.rpt` files are acceptable at this stage; structured `impl_summary.json` comes in LH2-H).
4. Short video (30–60 s) of the complete end-to-end system running: cooperative-target ping-pong, Threat forwarding, Nexys A7 seven-segment display updating with Beacon IDs and the frame counter, and a button-hold demonstration showing the display going silent then resuming.
5. One-page Module 1 retrospective: what worked, what didn't, what you would change before Module 2.

Glossary

CRC	Cyclic Redundancy Check	1, 10
FIFO	First In, First Out	2, 8, 13
FPGA	Field-Programmable Gate Array	1
RF	Radio Frequency	1
UART	Universal Asynchronous Receiver/Transmitter	1, 2
VHDL	VHSIC Hardware Description Language	1, 5, 17, 18